

LAB PROGRAMS

Name: S. Aarathana

Reg No: 1921260021

1. 8-PUZZLE

Aim

To implement the 8-Puzzle problem using Python and move the tiles to reach the goal state.

Procedure

1. Represent the puzzle using a list of 9 elements.
2. Use 0 to represent the blank space.
3. Find the position of the blank.
4. Ask the user to select a valid movement.
5. Swap the blank with the selected tile.
6. Continue until the goal state is reached.

Algorithm

1. Start with the initial puzzle.
2. Set the goal as 1 2 3 4 5 6 7 8 0.
3. Find the position of 0.
4. Check whether the selected movement is valid.
5. Swap the blank with the adjacent tile.
6. Repeat until the puzzle is solved.

Program

```
p = [1, 2, 3, 4, 0, 6, 7, 5, 8]
goal = [1, 2, 3, 4, 5, 6, 7, 8, 0]

while p != goal:
    print("\nCurrent Puzzle:")
    print(p[0], p[1], p[2])
    print(p[3], p[4], p[5])
    print(p[6], p[7], p[8])

    pos = p.index(0)

    print("\n1. Up")
    print("2. Down")
    print("3. Left")
    print("4. Right")

    choice = int(input("Enter move: "))

    if choice == 1 and pos >= 3:
        p[pos], p[pos-3] = p[pos-3], p[pos]
    elif choice == 2 and pos < 6:
        p[pos], p[pos+3] = p[pos+3], p[pos]
    elif choice == 3 and pos % 3 != 0:
        p[pos], p[pos-1] = p[pos-1], p[pos]
    elif choice == 4 and pos % 3 != 2:
        p[pos], p[pos+1] = p[pos+1], p[pos]
    else:
        print("Invalid move")

print("\nPuzzle Solved!")
```

```
print(1, 2, 3)
print(4, 5, 6)
print(7, 8, 0)
```

Output

Current Puzzle:

```
1 2 3
4 0 6
7 5 8
```

Enter move: 2

Current Puzzle:

```
1 2 3
4 5 6
7 0 8
```

Enter move: 4

Puzzle Solved!

```
1 2 3
4 5 6
7 8 0
```

Result

Thus, the 8-Puzzle problem was successfully implemented and solved.

2. 8-QUEEN

Aim

To implement the 8-Queen problem using Python by placing eight queens on a chessboard without attacking each other.

Procedure

1. Create an 8×8 chessboard.
2. Place one queen in each row.
3. Get the column position from the user.
4. Check whether the position is safe.
5. Check column and diagonal conflicts.
6. Continue until all eight queens are placed.

Algorithm

1. Initialize an empty 8×8 board.
2. Start placing a queen row by row.
3. Take the column position as input.
4. Check for same-column and diagonal conflicts.
5. Place the queen if the position is safe.
6. Repeat until all 8 queens are placed.

Program

```
n = 8
board = [-1] * n

for row in range(n):
    print("\nBoard:")

    for i in range(n):
        for j in range(n):
            if board[i] == j:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()

    col = int(input("Enter column (1-8): ")) - 1
    safe = True

    for i in range(row):
        if board[i] == col:
            safe = False
        if abs(board[i] - col) == abs(i - row):
            safe = False

    if safe:
        board[row] = col
        print("Queen placed!")
    else:
        print("Invalid position! Try again.")
        break

print("\nFinal Board:")

for i in range(n):
    for j in range(n):
```

```

        if board[i] == j:
            print("Q", end=" ")
        else:
            print(".", end=" ")
    print()

```

Output

```

Q . . . . . . . .
. . . . . Q . . .
. . . . . . . . Q
. . . . . Q . . .
. . Q . . . . . .
. . . Q . . . . .
. Q . . . . . . .
. . . . . . Q .

```

Result

Thus, the 8-Queen problem was successfully implemented by placing queens without attacking each other.

3. WATER JUG

Aim

To implement the Water Jug problem using Python and obtain the required quantity of water.

Procedure

1. Take two jugs of 4 litres and 3 litres.
2. Initially both jugs are empty.
3. Display the available operations.
4. Perform fill, empty and pour operations.
5. Display the current amount in both jugs.
6. Continue until one jug contains 2 litres.

Algorithm

1. Initialize both jugs to zero.
2. Set the capacities as 4L and 3L.
3. Select an operation.
4. Perform the selected operation.
5. Check whether 2L is obtained.
6. Repeat until the goal is reached.

Program

```
a = 0
b = 0

print("Jug 1 = 4 litres")
print("Jug 2 = 3 litres")
print("Goal = 2 litres")

while a != 2 and b != 2:
    print("\n1. Fill Jug 1")
    print("2. Fill Jug 2")
    print("3. Empty Jug 1")
    print("4. Empty Jug 2")
    print("5. Pour Jug 1 into Jug 2")
    print("6. Pour Jug 2 into Jug 1")

    choice = int(input("Enter choice: "))

    if choice == 1:
        a = 4
    elif choice == 2:
        b = 3
    elif choice == 3:
        a = 0
    elif choice == 4:
        b = 0
    elif choice == 5:
        x = min(a, 3 - b)
        a = a - x
        b = b + x
    elif choice == 6:
        x = min(b, 4 - a)
        b = b - x
        a = a + x
    else:
        print("Invalid choice")
```

```
print("Jug 1 =", a, "litres")
print("Jug 2 =", b, "litres")

print("\nGoal Reached!")
```

Output

```
Jug 1 = 4 litres
Jug 2 = 3 litres
Goal = 2 litres
```

```
Enter choice: 2
Jug 1 = 0 litres
Jug 2 = 3 litres
```

```
Enter choice: 6
Jug 1 = 3 litres
Jug 2 = 0 litres
```

```
Enter choice: 2
Jug 1 = 3 litres
Jug 2 = 3 litres
```

```
Enter choice: 6
Jug 1 = 4 litres
Jug 2 = 2 litres
```

```
Goal Reached!
```

Result

Thus, the Water Jug problem was successfully implemented and the required 2 litres of water was obtained.

4. CRYPT-ARITHMETIC

Aim

To implement a Crypt-Arithmetic problem in Python where each letter represents a unique digit.

Procedure

1. Assign digits from 0 to 9 to the letters.
2. Ensure that different letters have different digits.
3. Ensure that the first letters are not zero.
4. Convert the words into numbers.
5. Check whether the arithmetic equation is correct.
6. Display the valid solution.

Algorithm

1. Start with the letters S, E, N, D, M, O, R, Y.
2. Try different digits for each letter.
3. Check that all digits are different.
4. Form the numbers SEND, MORE and MONEY.
5. Check whether $SEND + MORE = MONEY$.
6. Display the solution.

Program

```
for S in range(1, 10):
    for E in range(10):
        for N in range(10):
            for D in range(10):
                for M in range(1, 10):
                    for O in range(10):
                        for R in range(10):
                            for Y in range(10):

                                if len({S,E,N,D,M,O,R,Y}) == 8:
                                    SEND = 1000*S + 100*E + 10*N + D
                                    MORE = 1000*M + 100*O + 10*R + E
                                    MONEY = 10000*M + 1000*O + 100*N + 10*E + Y

                                    if SEND + MORE == MONEY:
                                        print("SEND =", SEND)
                                        print("MORE =", MORE)
                                        print("MONEY =", MONEY)
```

Output

```
SEND = 9567
MORE = 1085
MONEY = 10652
```

```
  9567
+ 1085
-----
10652
```

Result

Thus, the Crypt-Arithmetic problem was successfully solved by assigning unique digits to the letters.

5. MISSIONARIES AND CANNIBALS

Aim

To implement the Missionaries and Cannibals problem using Python and find a valid sequence of states to move all people across the river.

Procedure

1. Start with 3 missionaries and 3 cannibals on the left side.
2. A boat can carry one or two people.
3. Generate possible boat movements.
4. Check whether the resulting state is valid.
5. Store valid states and continue searching.
6. Display the path from the starting state to the goal state.

Algorithm

1. Set the initial state as (3,3,1).
2. Generate possible combinations of missionaries and cannibals.
3. Move them according to the boat position.
4. Check that missionaries are never outnumbered by cannibals.
5. Continue until (0,0,0) is reached.
6. Display the solution path.

Program

```
states = [(3, 3, 1)]
parent = {}
i = 0

while i < len(states):
    m, c, boat = states[i]
    i = i + 1

    for x in range(3):
        for y in range(3):
            if x + y >= 1 and x + y <= 2:

                if boat == 1:
                    nm = m - x
                    nc = c - y
                    nb = 0
                else:
                    nm = m + x
                    nc = c + y
                    nb = 1

                if nm >= 0 and nc >= 0 and nm <= 3 and nc <= 3:

                    if (nm == 0 or nm >= nc) and (3-nm == 0 or 3-nm >= 3-nc):
                        new = (nm, nc, nb)

                        if new not in states:
                            states.append(new)
                            parent[new] = (m, c, boat)

current = (0, 0, 0)
path = []

while current != (3, 3, 1):
```

```
    path.append(current)
    current = parent[current]

path.append((3, 3, 1))
path.reverse()

print("Solution:")

for state in path:
    print(state)
```

Output

```
Solution:
(3, 3, 1)
(3, 1, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(0, 2, 1)
(0, 0, 0)
```

Here:

(m, c, boat) means:

m = missionaries on the left

c = cannibals on the left

1 = boat on left

0 = boat on right

Result

Thus, the Missionaries and Cannibals problem was successfully implemented and a valid solution path was obtained.